

Utilizzando il criterio di costo uniforme, il costo computazionale è $O(\log n)$. Utilizzando il criterio di costo logaritmico, supponiamo che n sia rappresentato con k bit, $n = O(2^k)$. Le operazioni sugli interi (confronto, somma, divisione per due) costano $O(k)$; l'elevamento al quadrato costa $O(k^2)$ con l'algoritmo banale. Il costo è quindi pari a $T(n) = O(k^2 \log n) = O(\log^3 n)$.

```

int isSquareRec(int n, int i, int j)
    if i == j then
        | return i
    else
        | int m =  $\lfloor (i + j) / 2 \rfloor$ 
        | if n == m2 then
        | | return m
        | else if m2 > n then
        | | return isSquareRec(n, i, m)
        | else
        | | return isSquareRec(n, m + 1, j)

```

```

boolean isSquare(int n)
    int v = squareRec(n, 1, n)
    return n == v2;

```

Si noti tuttavia che un semplice algoritmo come questo:

```

isSquare(int n)
    int i = 1
    while i2 < n do
        | i = i + 1
    return i2 == n

```

ha costo $O(\sqrt{n})$ misurato con il criterio di costo uniforme; ha costo $O(\log^2 n \cdot \sqrt{n})$ se misurato con criterio di costo logaritmico.

Esercizio B1 – PrintBitsK – Punti ≥ 8

Risolviamo il problema utilizzando la tecnica di backtrack. La funzione ricorsiva prende in input:

- il vettore S dove verranno memorizzate le stringhe binarie;
- l'indice i che rappresenta la posizione dove scrivere il prossimo bit (inizialmente n , quando raggiunge zero la stringa è terminata);
- il valore c_1 che indica quanti bit 1 consecutivi è ancora possibile inserire (inizialmente k , quando raggiunge zero non è più possibile inserire bit 1);
- il valore k , costante, necessario per riportare c_1 a k quando si inserisce un bit 0.

Se $i = 0$, abbiamo riempito tutti i bit (partendo dal fondo) e quindi possiamo stampare la stringa binaria. Altrimenti, si prova a inserire 0 (sempre possibile), passando il valore k nel parametro c_1 della chiamata ricorsiva; se c_1 è maggiore di zero, è possibile inserire anche il valore 1, riducendo di 1 il valore c_1 .

Nel caso $k = n$, è possibile stampare tutte le stringhe binarie di n elementi, che sono 2^n , ognuna delle quali viene stampata in tempo $O(n)$. Il costo totale è quindi $O(n2^n)$.

```
printRec(int [] S, int i, int c1, int k)
```

```
if i == 0 then
    print(S)
else
    S[i] = 0
    printRec(S, i - 1, k, k)
    if c1 > 0 then
        S[i] = 1
        printRec(S, i - 1, c1 - 1, k);
```

```
printBits(int n, int k)
```

```
int[] S = new int[1...n]
printRec(S, n, k, k);
```

Esercizio B2 – Sequenza k -limitata – Punti ≥ 10

Questo problema è risolubile in maniera simile all'Esercizio B3 del 04/09/2020, anche se prende le definizioni dall'esercizio B3 del 3/7/2020.

Si utilizza la tecnica backtrack utilizzando uno stack. In questa versione, partiamo dal fondo. Se l'elemento che si sta considerando dista più di k dall'ultimo valore inserito nello stack (oppure se lo stack è vuoto), abbiamo due possibilità: si può inserire oppure no nello stack. Se invece è più grande, viene saltato.

Nel caso di un vettore V in cui $\max(V) - \min(V) \leq k$, ognuna delle 2^n sottosequenze deve essere stampata, con costo $\Theta(n)$. La complessità computazionale è quindi $O(n \cdot 2^n)$.

```
printKSequence(int[] A, int n, int k)
```

```
STACK S = Stack()
printRec(A,k,n,S)
```

```
printRec(int[] A, int k, int i, STACK S)
```

```
if i==0 then
    print S                                     % LIFO order
else
    if S.isEmpty() or |A[i] - S.top()| > k then
        S.push(A[i])
        printRec(A, k, i - 1, S)
        S.pop()
    printRec(A, k, i - 1, S)
```

Esercizio B3 – Somme Palindrome – Punti ≥ 12

Le somme palindrome di lunghezza pari hanno lo stesso valore all'inizio e alla fine della sequenza. È quindi sufficiente considerare tutti i possibili valori compresi fra 1 e $n/2$, e sottrarli due volte al valore n , considerando quindi tutti le somme palindrome del valore $n - 2i$ con lunghezza pari.

Per effettuare il calcolo, possiamo utilizzare la programmazione dinamica. Sia $DP[i]$ il numero di somme palindrome di i con lunghezza pari. DP può essere calcolato in questo modo, per valori pari:

$$DP[i] = \begin{cases} 1 & i = 0 \\ \sum_{j=1}^{i/2} DP[i - 2j] & i > 0 \end{cases}$$

È facile tradurre questo algoritmo in un algoritmo di programmazione dinamica bottom-up.

```
int countSumEven(int n)
int[] DP = new int[0...n]
DP[0] = 1
for i = 2 to n step 2 do
    DP[i] = 0
    for j = 1 to i/2 do
        DP[i] = DP[i] + DP[i - 2j]
return DP[n]
```

L'algoritmo ha complessità $O(n^2)$.